FINAL PROJECT REPORT

Harnessing Earnings Reports for Stock Predictions: Reproducing a QLoRA-Enhanced LLM Approach and Testing Numerical Baselines

Stage 1: Reproduction of QLoRA fine-tuning of Llama-3-8B
Stage 2: Lightweight numerical models (Logistic Regression and Random Forest)

Original paper: H. Ni et al., arXiv:2408.06634v2, 2024

Abstract

This project reproduces and extends Ni et al. (2024), *Harnessing Earnings Reports for Stock Predictions: A QLoRA-Enhanced LLM Approach*. The paper fine-tunes Llama-3-8B-Instruct with 4-bit QLoRA to predict whether a stock will close above its opening price on the first trading day after an earnings call, and reports a test accuracy of 0.573 and an MCC of 0.154.

Stage 1 rebuilt the dataset from free public sources (8,271 earnings calls from 481 S&P 500 companies, January 2020 to June 2024) and fine-tuned the same model with the paper's hyper-parameters on a free Kaggle T4 GPU, using a strict chronological train/validation/test split. Our fine-tuned models reached test accuracies of 0.523 (Base) and 0.517 (Full), with ROC-AUC between 0.51 and 0.53, which is close to chance. Our zero-shot Llama-3 baseline (0.496) matched the paper's GPT-4 baseline (0.494).

Stage 2 asked whether the numerical information in the prompts carries a signal on its own. We extracted the 12 financial growth metrics and 6 market variables from the prompts without loss, built a leakage-safe preprocessing pipeline, and trained Logistic Regression (main model) and Random Forest (optional model), selected only by validation ROC-AUC. On the same 946 test events, all Stage 2 models had ROC-AUC between 0.510 and 0.527. None of 24 paired bootstrap comparisons against Stage 1 models, Always Short, or each other was significant (all $p \geq 0.14$). The conclusion is a careful null result: with our data and target, neither the LLM nor the numerical models found a reliable next-day signal. Stage 2 reached the same level as the LLM at a tiny fraction of its compute cost.

Contents

Abstract 1

1 Application and Problem Statement 3

2 Dataset Details 4

3 Original Research Method 6

4 Stage 1 – Implementation 8

5 Stage 1 – Results 10

6 Stage 2 – Proposed Modifications 11

7 Stage 2 – Experimental Results 14

8 Conclusions 18

9 References 19

Appendix A How to Reproduce 19

List of Figures and Tables

Figure 1. Block diagram of the original method (Ni et al., 2024) 6
Figure 2. Block diagram of the modified method (Stage 1 path and new Stage 2 path) 12
Figure 3. Test metrics of all models with 95% bootstrap confidence intervals 14
Figure 4. ROC curves on the test set (Base and Full) 15
Figure 5. Paired bootstrap differences in test ROC-AUC 16
Figure 6. Standardized Logistic Regression coefficients 17

Table 1. Data sources: paper versus this project 4
Table 2. Dataset size and filtering 4
Table 3. Fields of one dataset record 5
Table 4. Chronological train / validation / test split 5
Table 5. Training settings reported in the paper 7
Table 6. Stage 1 software, hardware and model 8
Table 7. Stage 1 QLoRA settings: paper versus ours 8
Table 8. Stage 1 experiments that were run 9
Table 9. Stage 1 test results: paper versus reproduction 10
Table 10. Stage 2 modifications: what, why and how 11
Table 11. Stage 2 input features 12
Table 12. Stage 2 model settings and search grids 13
Table 13. Validation search and selected settings 14
Table 14. Stage 2 test results (946 events) 14
Table 15. All models on the same 946 test events 15
Table 16. Paired bootstrap tests of ROC-AUC differences 16

1 Application and Problem Statement

1.1 Application

Every quarter, listed companies publish their results and hold an **earnings call**, in which management presents the numbers and answers analysts' questions. These events move prices strongly, and investors, analysts and funds try to anticipate the market's reaction. The application studied here is **post-earnings stock direction prediction**: given what is known at the time of the call, predict whether the stock will rise or fall during the next trading session.

1.2 Why it matters

- **Investors and traders** take long or short positions around earnings. A model that is right even slightly more often than chance could be valuable.
- **Information overload.** An earnings call transcript averages about 11,500 Llama-3 tokens in our data (12,375 in the paper). A human cannot read thousands of calls each season, but a language model can.
- **Two kinds of information.** The numbers (growth in revenue, profit, cash flow) and the text (management tone, guidance, answers to analysts) may both matter. Classical models use only numbers; text models such as FinBERT handle only short passages. The paper proposes a large language model (LLM) that reads both.

1.3 Problem addressed

The task is binary classification. For each earnings event the model receives company financial information and the call transcript and must answer one word:

$$y = \text{Long if } \text{Close}_{\text{next session}} > \text{Open}_{\text{next session}}, \text{ otherwise Short} \tag{1}$$

The “next session” is the first trading session after the call: the same day for calls before 09:30 US Eastern time, and the following trading day otherwise. Because the label is measured from the open to the close, the overnight price jump that follows the announcement is *not* part of the label.

1.4 Objectives of this project

1. **Stage 1 (reproduction):** rebuild the paper's dataset from free sources, fine-tune Llama-3-8B-Instruct with 4-bit QLoRA using the paper's settings, and compare our results with the published ones (accuracy 0.550 Base and 0.573 Full).
2. **Stage 2 (modification):** test whether the numerical features inside the prompts can predict the direction on their own, using lightweight and interpretable models (Logistic Regression, Random Forest), and compare them statistically with every Stage 1 model on exactly the same test events.

2 Dataset Details

2.1 Sources

The paper did not release its data or code, and its source (the Financial Modeling Prep API) puts transcripts, analyst grades and earnings surprises behind paid plans. We therefore rebuilt the same recipe from free sources (Table 1).

Component	Paper	This project
Company universe	501 S&P 500 companies	S&P 500 list as of 10 Aug 2024 (GitHub `datasets/s-and-p-500-companies`, commit 9b6713a3, includes SEC CIK numbers)
Earnings call transcripts	Financial Modeling Prep	Hugging Face dataset `kurry/sp500_earnings_transcripts`; calls shorter than 2,000 characters dropped as empty or broken
Financial growth (YoY)	Financial Modeling Prep	SEC EDGAR `companyfacts` API (XBRL). Quarterly 3-month values; Q4 and cash-flow items derived from year-to-date differences; earliest-filed value used to avoid later restatements
Past-week returns	Financial Modeling Prep	Yahoo Finance (`yfinance`): 5-trading-day return of the stock, SPY, QQQ and the Dow (^DJI), ending at the last close before the call
EPS surprise	Financial Modeling Prep	Yahoo Finance earnings dates: (actual − estimate) / |estimate|
Analyst grade	Financial Modeling Prep	Yahoo Finance upgrades/downgrades: most frequent of buy/hold/sell in the 30 days before the call
Prices for the label	Financial Modeling Prep	Yahoo Finance daily open and close

Table 1. Data sources: paper versus this project.

2.2 Size and filtering

Item	This project	Paper
Companies with transcripts	497	–
Earnings calls in period (Jan 2020 – Jun 2024)	8,729	–
Dropped: no price data or label	264	–
Dropped: fewer than 3 financial growth metrics	29	–
Dropped: no EPS data	165	–
Final events	**8,271**	**8,556**
Final companies	**481**	**501**
Share of the paper's size	96.7%	100%
Label balance	49.1% Long / 50.9% Short	not reported
Average transcript length (Llama-3 tokens)	about 11,500	12,375

Table 2. Dataset size and filtering.

Companies were lost mainly because free price history is unavailable for firms that were acquired or delisted after August 2024 (for example ANSS, CTLT, HES, JNPR, WBA), or because no transcript exists (for example BRK.B, NVR).

2.3 Data format

The dataset is a compressed JSON-lines file, `stage1/1_data/llm_data.jsonl.gz`, with one record per earnings event (Table 3). A second file, `llm_data_sum.jsonl.gz`, has the same records with the transcript replaced by an extractive summary (Section 2.4).

Field	Content
`event_id`	Ticker and call date, e.g. `NVDA_2024-02-21` (unique key)
`ticker`, `date`	Company symbol and call date
`split`	`train`, `val` or `test` (Section 2.5)
`label`	`Long` or `Short`, from Equation (1)
`instruction`	The task instruction given to the LLM
`base_head`	Company, date and the 12 financial growth sentences
`full_head`	`base_head` plus market returns, analyst grade and EPS surprise sentences
`transcript`	Earnings call text, stored up to 40,000 characters

Table 3. Fields of one dataset record.

Following the paper, every number is **textualized**, that is, written as an English sentence. The Full prompt head for one real event looks like this (shortened):

```
Company: Lennar Corporation (LEN)
Earnings call date: 2020-01-08
Financial metrics growth:
Compared to the same quarter last year, Revenue grew by 7.9%.
Compared to the same quarter last year, Net Income declined by 15.3%.
...  (12 metrics in total)
Market performance:
In the past week, the company's stock (LEN) went up by 3.2%.
In the past week, SPY went up by 0.8%.   (also QQQ and DOW)
Analyst grades:
In the past 30 days, most grading companies suggest buying this stock.
Earnings surprise:
The company's reported earnings per share were 12.1% higher than analysts' consensus estimates.
Earnings call transcript: ...
```

2.4 Preprocessing

1. **Year-over-year growth** is computed for 12 metrics: Revenue, Gross Profit, Operating Income, Net Income, EPS (diluted), Operating Cash Flow, Capital Expenditure, Total Assets, Total Liabilities, Shareholders' Equity, Cash and Cash Equivalents, and Free Cash Flow (= Operating Cash Flow − Capital Expenditure). A metric that a company does not report (for example Gross Profit for banks) is left out of its prompt; events need at least 3 metrics.
2. **Label timing:** a call before 09:30 ET is labelled with that day's session; a call at or after 09:30 ET with the next session.
3. **Transcript length.** Transcripts are stored up to 40,000 characters, which cut 88% of calls, removing about 23% of each call on average (mostly the end of the Q&A). The free GPU allowed only 3,072 tokens per prompt, so in the first runs the model saw only about the first 12,000 characters.
4. **Extractive summarization (Stage 1b).** To let the model see the whole call, each transcript was replaced by a deterministic extractive summary of about 8,500 characters (about 2,200 tokens). Operator turns and boilerplate are removed; the call is split into prepared remarks and Q&A; each sentence is scored by finance keywords (guidance, outlook, margin, demand, ...) and by numbers, percentages and dollar amounts; each half of the call gets half of the budget; and the selected sentences are kept in their original order. 99.3% of summaries contain Q&A sentences, and no prompt is truncated. Script: `summarize_transcripts.py`.

2.5 Train / validation / test split

Many companies report on the same day and share that day's market move. A random split would put same-day events in both training and test data and let a model learn the market direction of a test day, which inflates accuracy. We therefore used a strict **chronological split** (Table 4). The same split is used in every stage, and the test set was scored once per final model.

Partition	First call	Last call	Events	Long	Short	Long %	Use
Train	2020-01-03	2023-06-30	6,387	3,161	3,226	49.5	Fitting
Validation	2023-07-13	2023-12-21	938	441	497	47.0	Model selection
Test	2024-01-04	2024-06-27	946	457	489	48.3	Final evaluation only
All	2020-01-03	2024-06-27	**8,271**	4,059	4,212	49.1	

Table 4. Chronological train / validation / test split. Because 51.7% of test events are Short, always answering Short gives 0.517 test accuracy.

3 Original Research Method

Ni et al. (2024) turn all inputs into text and fine-tune a pre-trained LLM to answer one word. Figure 1 shows the pipeline as described in the paper.

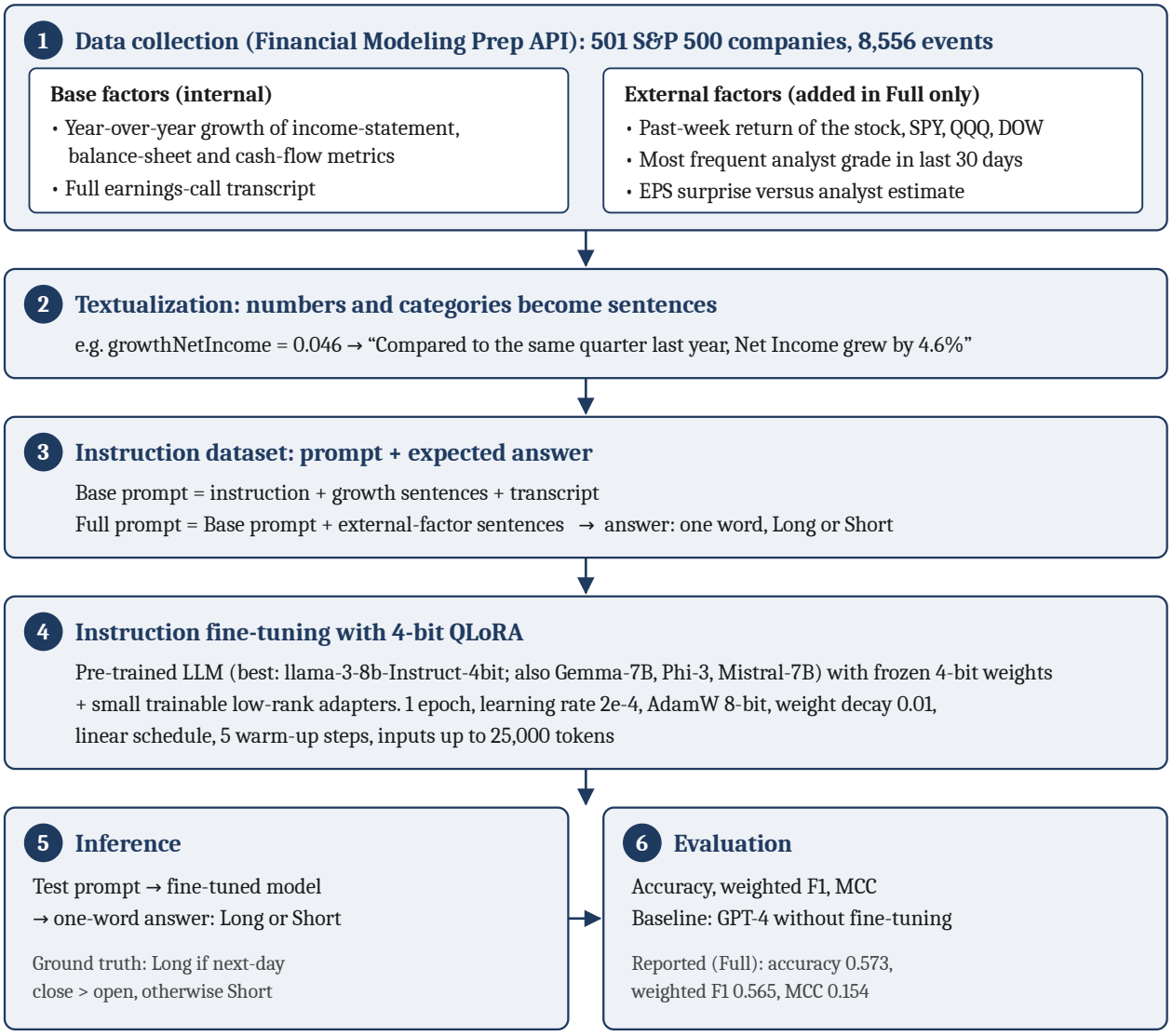


Figure 1. Block diagram of the original method (Ni et al., 2024), drawn from the paper's description.

3.1 Components

1. **Data collection.** For each earnings event, the authors collect *base factors* (financial growth and the transcript), which describe the company itself, and *external factors* (market returns, analyst grades, EPS surprise), which describe its environment. Rows with missing values are removed, leaving 8,556 events.
2. **Textualization.** An LLM reads text, so every number is converted into a short sentence. This keeps the meaning of each number (its name, sign and size) inside the prompt.
3. **Instruction dataset.** Each event becomes a pair: an instruction-style prompt and the expected answer (Long or Short). Two versions are built, **Base** and **Full**, to measure the value of the external factors.
4. **QLoRA fine-tuning.** The base model's weights are stored in 4-bit precision and kept frozen. Small low-rank matrices (LoRA adapters) are added to the layers and are the only trained parameters, so an 8-billion-parameter model can be fine-tuned on one GPU. For a frozen weight matrix W, the adapted layer computes

$$h = Wx + (\alpha/r)\cdot BAx, \quad A \in \mathbb{R}^{r\times k}, B \in \mathbb{R}^{d\times r}, r \ll \min(d,k) \tag{2}$$

   where only A and B are trained, r is the rank and α a scaling constant.
5. **Inference.** A test prompt is given to the fine-tuned model, which generates one word that is mapped to the label.

6. **Evaluation.** The paper reports accuracy, weighted F1 and the Matthews correlation coefficient (MCC), and compares several 4-bit models with GPT-4 used without fine-tuning.

3.2 Training settings stated in the paper

Setting	Value in the paper
Best model	llama-3-8b-Instruct-4bit
Epochs / learning rate	1 / 2e-4, linear schedule, 5 warm-up steps
Optimizer / weight decay	AdamW 8-bit / 0.01
Maximum sequence length	25,000 tokens
Batch size, LoRA rank, data split, hardware	Not reported
Results (Base): Acc / wF1 / MCC	0.550 / 0.533 / 0.104
Results (Full): Acc / wF1 / MCC	0.573 / 0.565 / 0.154
GPT-4 baseline (Base; Full)	0.363 / 0.482 / 0.023; 0.494 / 0.512 / 0.031

Table 5. Training settings and results reported in the paper. Unreported settings had to be chosen by us.

4 Stage 1 – Implementation

4.1 Tools and environment

Item	Used in this project
Hardware	Kaggle free GPU: NVIDIA Tesla T4 (about 15 GB), 12-hour session limit, 30 GPU hours per week
Base model	`unsloth/llama-3-8b-Instruct-bnb-4bit` (Llama-3-8B-Instruct pre-quantized to 4 bits)
Fine-tuning library	Unsloth 2026.10.1 (QLoRA), with Hugging Face Transformers 4.57.6 and TRL 0.22.2
Deep learning framework	PyTorch 2.11 (CUDA 12.8), as installed on Kaggle
Data building	Python with pandas, `requests` (SEC EDGAR), `yfinance`, Hugging Face `datasets`
Evaluation	scikit-learn metrics, NumPy bootstrap
Code	`stage1/2_code/dataset_builder/step01–step05` (dataset), `llm_stage1.py` (training and scoring), notebook `1_Train_and_Evaluate.ipynb`

Table 6. Stage 1 software, hardware and model.

4.2 Model and QLoRA settings

We used the paper's hyper-parameters where they are stated and the Unsloth defaults (which the paper's values match) elsewhere. Two settings had to be reduced to fit the free T4 GPU: the prompt length and the number of training events (Table 7).

Setting	Ours	Paper
LoRA rank / alpha / dropout	16 / 16 / 0	not reported
LoRA target layers	all attention and MLP projections (q, k, v, o, gate, up, down)	not reported
Trainable parameters	41.9 million of 8.07 billion (0.52%)	not reported
Epochs / learning rate / schedule	1 / 2e-4 / linear, 5 warm-up steps	same
Optimizer / weight decay	adamw_8bit / 0.01	same
Effective batch size	8 (batch 1 × gradient accumulation 8)	not reported
Maximum sequence length	3,072 tokens	25,000
Training events	3,000 from the training period, balanced 50% Long / 50% Short (375 optimizer steps)	not reported
Validation / test events	500 (subset of validation period) / 946	not reported
Loss	cross-entropy on the answer tokens only (`train_on_responses_only`)	not reported

Table 7. Stage 1 QLoRA settings. Highlighted rows differ from the paper because of the free GPU. A 10-step timing run measured about 77 s per step, so 6,387 events would have needed about 17 hours, above Kaggle's 12-hour limit.

4.3 Prompt and training

Each training example uses the Llama-3 chat format: the system/user part holds the instruction, the Base or Full head, and the transcript (or its summary); the assistant part holds the answer `Long` or `Short`. The loss is computed only on the answer, so the model is trained to produce the right word and not to copy the prompt. Training events were balanced because a pilot run on 400 unbalanced events predicted Long for every event.

4.4 Scoring and the fp32 fix

Instead of only reading the generated word, we compute a probability from the model's next-token scores (logits) z for the two answer tokens (token IDs 6720 for “Long” and 12755 for “Short”):

$$\mathrm{P(Long)} = \exp(z_{\mathrm{Long}}) \,/\, [\exp(z_{\mathrm{Long}}) + \exp(z_{\mathrm{Short}})], \quad \text{prediction} = \text{Long if } \mathrm{P(Long)} \geq 0.5 \tag{3}$$

This gives the same decision as the paper's one-word answer, but also allows ROC-AUC. In the first runs the last step was computed in half precision (fp16). Both logits were about 20 and nearly equal, and fp16 cannot represent such small gaps: P(Long) took only 7 distinct values and 287 of 946 test events were exact 0.500 ties, all counted as Long. This made weighted F1 look about 7 points better than it really was. We re-computed the last step in single precision (fp32), which removed all ties. All Stage 1 results in this report use fp32.

4.5 Evaluation metrics

With TP, TN, FP, FN the confusion-matrix counts (Long = positive class):

$$\text{Accuracy} = (TP + TN) / N \tag{4}$$

$$MCC = (TP \cdot TN - FP \cdot FN) / \sqrt{[(TP+FP)(TP+FN)(TN+FP)(TN+FN)]} \tag{5}$$

Weighted F1 averages the F1 score of each class, weighted by the number of true events in that class. MCC ranges from −1 to 1; it is 0 for random guessing and also when a model predicts only one class. **ROC-AUC** is the probability that a random Long event receives a higher P(Long) than a random Short event; 0.5 means no ranking ability. **95% confidence intervals** come from 2,000 bootstrap resamples of the test events (seed 42).

4.6 Experiments run

#	Experiment	Data	Outcome
1	Timing run (10 steps)	Base	about 77 s per step → training capped at 3,000 events
2	Pilot, 400 events, validation only	Base	predicted Long for every event → balanced training data
3	QLoRA, cut transcripts (7.5 h)	Base	train loss 0.379; fp32 re-score: accuracy 0.521, AUC 0.519
4	QLoRA, cut transcripts (8.2 h)	Full	predicted Short for all 946 test events
5	QLoRA, summarized transcripts (5.5 h + 0.9 h scoring)	Base	**final Base model** (Table 9)
6	QLoRA, summarized transcripts (6.3 h)	Full	**final Full model** (Table 9)
7–8	Zero-shot Llama-3 (no fine-tuning, about 1 h each)	Base, Full	baseline comparable to the paper's GPT-4 (Table 9)

Table 8. Stage 1 experiments that were run (about 33 GPU hours in total). Highlighted rows are the final runs reported in Section 5.

5 Stage 1 – Results

5.1 Paper versus reproduction

Model	Data	Accuracy [95% CI]	Weighted F1	MCC	ROC-AUC [95% CI]	Predicts Long
Reported in the paper						
Llama-3-8B QLoRA	Base	0.550	0.533	0.104	–	–
Llama-3-8B QLoRA	Full	0.573	0.565	0.154	–	–
GPT-4, no fine-tuning	Base	0.363	0.482	0.023	–	–
GPT-4, no fine-tuning	Full	0.494	0.512	0.031	–	–
Our reproduction (946 test events, Jan–Jun 2024)						
Llama-3-8B QLoRA	Base	0.523 [0.493, 0.555]	0.480	0.033	0.532 [0.498, 0.568]	21.4%
Llama-3-8B QLoRA	Full	0.517 [0.485, 0.550]	0.352	0.000	0.510 [0.473, 0.546]	0.0%
Llama-3-8B, no fine-tuning	Base	0.511 [0.479, 0.542]	0.497	0.034	0.509 [0.474, 0.546]	66.4%
Llama-3-8B, no fine-tuning	Full	0.496 [0.462, 0.529]	0.481	0.003	0.507 [0.470, 0.544]	67.0%
Always Short (reference)	–	0.517 [0.485, 0.550]	0.352	0.000	0.500	0.0%

Table 9. Stage 1 test results: paper versus reproduction. The paper reports no confidence intervals and no ROC-AUC. On the 500 validation events, ROC-AUC of our four runs was 0.482 to 0.497.

5.2 Comparison

- **Base.** Our accuracy (0.523) is 2.7 points below the paper's 0.550, but the paper's value lies inside our 95% interval [0.493, 0.555], so the difference could be sampling noise. Our MCC (0.033) is much lower than the paper's (0.104).
- **Full.** The paper's 0.573 lies above our interval. Our Full model predicted Short for every test event, so it is identical to the Always Short reference (MCC 0).
- **No fine-tuning.** Our zero-shot Llama-3 on Full (0.496) matches the paper's GPT-4 on Full (0.494). This part of the paper is reproduced.
- **Full versus Base.** The paper found Full better by about 2 accuracy points. We did not: the paired bootstrap AUC difference (Full − Base) of our fine-tuned models is −0.02 [−0.06, +0.02].

5.3 Observations

1. **The model learned the answer format but no signal.** Training loss settled at about 0.35 by step 40 and stayed there until the end (step 375). The loss is averaged over two answer tokens (the word and the end-of-turn token, which is trivial to predict), so a pure 50/50 guess gives $\ln 2 / 2 \approx 0.347$. The model therefore reached exactly coin-flip level on the word itself.
2. **Near-constant outputs.** P(Long) ranged only from 0.478 to 0.509 (Base) and 0.411 to 0.490 (Full). With almost the same probability for every event, a small offset decides the side: Full fell below 0.5 everywhere and always said Short; Base said Short 79% of the time.
3. **Summaries did not help.** Letting the model see the whole call (summaries) instead of only its beginning (cut transcripts) gave nearly the same results.
4. **Possible reasons for the gap to the paper** (our estimates, not proven): a shorter context (3,072 versus 25,000 tokens); fewer training events (3,000 versus about 6,800); free data sources, so features and transcripts are not identical; and the paper's unstated split — a random split would let same-day market moves leak between training and test.

6 Stage 2 – Proposed Modifications

6.1 Motivation

Stage 1 showed that the fine-tuned LLM stays at chance level. Two explanations are possible: (a) the inputs contain no usable signal for next-day direction, or (b) the signal exists, at least in the numbers, but the LLM cannot use it, because the numbers are a few sentences buried in about 2,000 tokens of transcript and the model only sees them as text. Stage 2 separates these explanations by giving the **same numbers**, as numbers, to classical models designed for tabular data, and comparing them with Stage 1 on **exactly the same test events**. Constraints kept from Stage 1: the same target, the same chronological split, no new data sources, no fine-tuning of an LLM, and no changes to Stage 1 files.

6.2 List of modifications

#	Modification	Why it was introduced	How it was implemented
M1	Lossless extraction of numbers from the prompts	The numbers must be exactly what the LLM saw (same rounding, same missing values), otherwise the comparison is unfair	`data_extraction.py`: regular expressions that mirror the Stage 1 sentence templates, stopping on any unparsed line. `verify_extraction.py` rebuilds every sentence from the numbers and compares it with the original: 0 mismatches in 8,271 events
M2	Replace the LLM and transcript by numerical features only	Tests explanation (b) directly: if the numbers alone predict the direction, the LLM was the bottleneck	The transcript is removed; models see only the numbers that were in the Base or Full prompt head
M3	Leakage-safe preprocessing	Growth rates have extreme values (e.g. +984%), many metrics are missing for some industries, and all statistics must come from training data only	`preprocessing.py`: scikit-learn `Pipeline` + `ColumnTransformer` fitted on the training split only (Section 6.4)
M4	Lightweight classifiers	Logistic Regression is simple, fast and interpretable (main model); Random Forest can capture non-linear effects and interactions (optional model)	`train_logistic_regression.py`, `train_random_forest.py`; small search grids, best setting chosen by validation ROC-AUC only; test scored once
M5	Stronger evaluation	Accuracy alone can hide one-class predictions; small differences need significance tests	`evaluate.py`, `compare.py`: accuracy, precision, recall, weighted F1, MCC, ROC-AUC, log loss, Brier score, share predicted Long, confusion matrix; bootstrap 95% CIs; paired bootstrap test of AUC differences on identical event IDs
M6	Interpretation and demo	To see which features the models rely on and to explain single predictions	Standardized LR coefficients, RF permutation importance on validation, and `predict_demo.py` (per-feature contributions for any event)

Table 10. Stage 2 modifications: what, why and how.

6.3 Modified block diagram

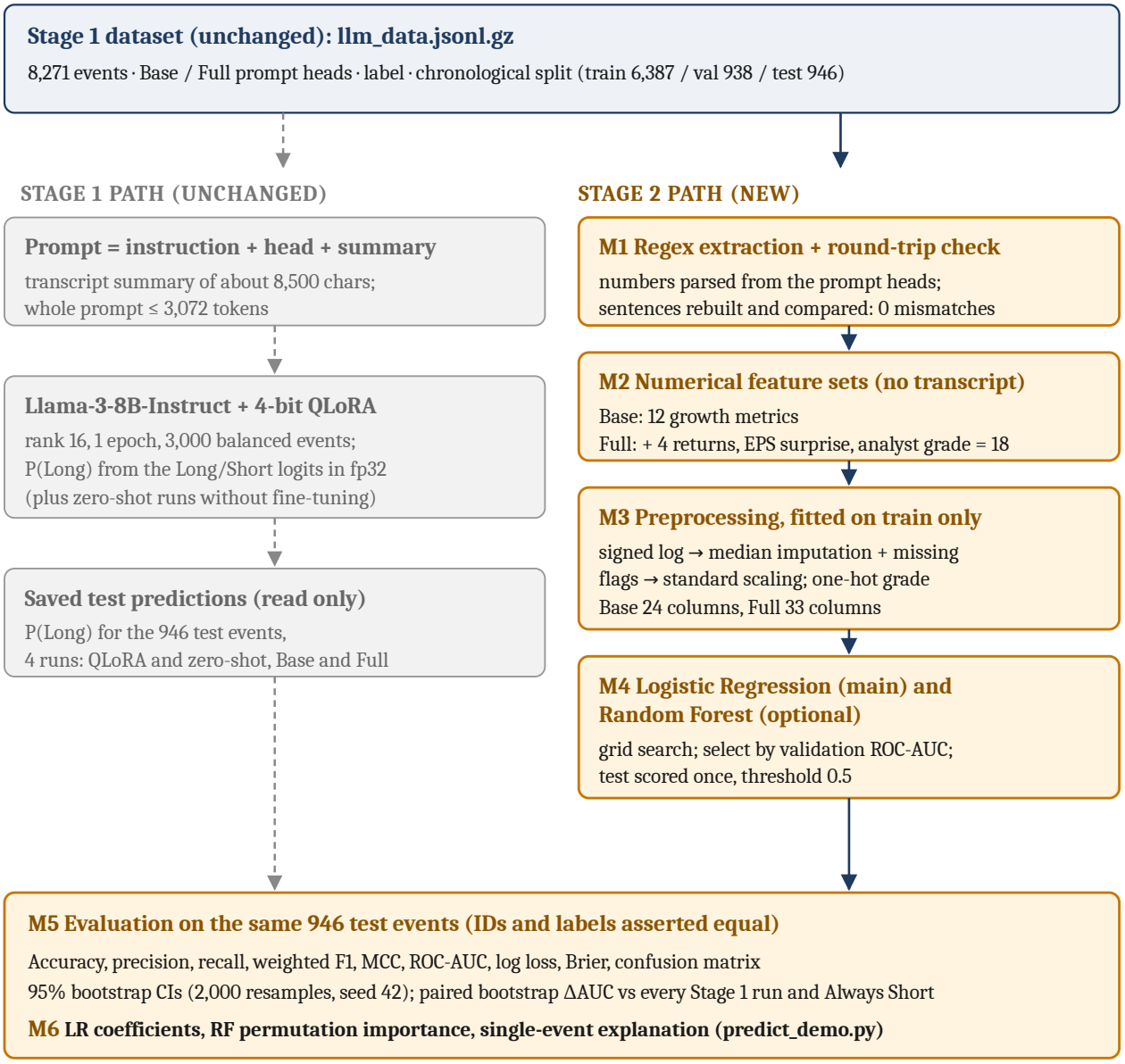


Figure 2. Modified method. Grey: Stage 1 path, unchanged and only read. Orange: new Stage 2 blocks (M1–M6 refer to Table 10).

6.4 Implementation details

Features

Group	Features	Missing (train)
Growth (Base and Full)	YoY % change of Revenue, Gross Profit, Operating Income, Net Income, EPS (diluted), Operating Cash Flow, Capital Expenditure, Total Assets, Total Liabilities, Shareholders' Equity, Cash and Equivalents, Free Cash Flow	0.2% to 60.1% (Gross Profit highest, Total Liabilities 32.7%)
Market (Full)	Past-week % return of the stock, SPY, QQQ, DOW	0%
EPS surprise (Full)	% difference between reported and estimated EPS	0%
Analyst grade (Full)	buy / hold / sell / none	"none" is a category

Table 11. Stage 2 input features: 12 for Base, 18 for Full. Identifiers (ticker, date, event ID) are never used as features.

Preprocessing (fitted on the 6,387 training events only)

1. **Signed log transform** for growth metrics and EPS surprise, which compresses extreme values while keeping the sign:

$$f(x) = \text{sign}(x) \cdot \ln(1 + |x| \,/\, 100) \tag{6}$$

For example +984% becomes 2.38 and −50% becomes −0.41. The transform has no fitted parameters.

2. **Median imputation with missing flags** for growth metrics: a missing value is replaced by the training median, and a 0/1 indicator column records that it was missing, because "not reported" may itself carry information (e.g. banks have no Gross Profit).
3. **One-hot encoding** of the analyst grade.
4. **Standardization** of all numeric columns with training statistics: $z = (x - \mu_{train}) / \sigma_{train}$.

Because the transformer is inside the model pipeline and fitted on training data only, no information from validation or test events is used to set medians, means or scales.

Models

Logistic Regression models the probability of Long as

$$P(\text{Long} \mid \mathbf{z}) = \sigma(b + \mathbf{w}^{T}\mathbf{z}) = 1 / (1 + \exp(-(b + \mathbf{w}^{T}\mathbf{z}))) \tag{7}$$

with L2 regularization controlled by C (smaller C = stronger shrinkage of $\mathbf{w}$ toward 0). Random Forest averages the votes of many decision trees, each trained on a bootstrap sample.

Model	Fixed settings	Search grid (validation)
Logistic Regression (main)	L2 penalty, lbfgs solver, max_iter 2,000, random_state 42	$C \in \{0.001, 0.01, 0.1, 1, 10\}$ × class_weight ∈ {none, balanced} = 10 settings
Random Forest (optional)	200 trees, min_samples_leaf 5, random_state 42	max_depth ∈ {5, 10, none} × class_weight ∈ {none, balanced} = 6 settings
Selection rule	Highest validation ROC-AUC; decision threshold fixed at 0.5; the test set is not used for any choice	

Table 12. Stage 2 model settings and search grids (scikit-learn 1.9.1). 2 models × 2 feature sets gives 32 validation runs.

Statistical comparison

For two models A and B scored on the same events, each bootstrap resample draws 946 test events with replacement and computes $\Delta = AUC_A - AUC_B$ on the same resample. Over 2,000 resamples this gives a 95% interval for Δ and a two-sided p-value:

$$p = 2 \cdot \min(\text{ share of } \Delta^{*} \le 0,\ \text{share of } \Delta^{*} \ge 0\ ) \tag{8}$$

Pairing matters: both models see the same easy and hard events in each resample, so the test is more sensitive than comparing two separate intervals.

7 Stage 2 – Experimental Results

7.1 Model selection on validation

Model	Data	Validation AUC range over grid	Selected setting	Selected validation AUC
Logistic Regression	Base	0.507 – 0.513	C = 0.001, no class weight	0.513
Logistic Regression	Full	0.486 – 0.492	C = 0.001, no class weight	0.492
Random Forest	Base	0.505 – 0.520	max_depth 10, balanced	0.520
Random Forest	Full	0.498 – 0.517	max_depth 10, balanced	0.517

Table 13. Validation search (938 events) and selected settings. Across all 32 runs, validation AUC stayed between 0.486 and 0.520.

Both Logistic Regression models chose the strongest regularization (C = 0.001), which shrinks all coefficients toward zero. This is what is expected when the features carry little linear signal: the less the model trusts the data, the better it does on validation.

7.2 Test results

Model	Data	Accuracy [95% CI]	Precision	Recall	Wtd F1	MCC [95% CI]	ROC-AUC [95% CI]	Log loss	Brier	Pred. Long
LR	Base	0.517 [0.487, 0.548]	0.500	0.333	0.501	0.023 [−0.036, 0.085]	0.527 [0.490, 0.563]	0.691	0.249	32.1%
LR	Full	0.539 [0.507, 0.571]	0.535	0.352	0.523	0.071 [0.010, 0.133]	0.523 [0.488, 0.561]	0.691	0.249	31.8%
RF	Base	0.515 [0.483, 0.544]	0.498	0.484	0.514	0.028 [−0.037, 0.086]	0.521 [0.484, 0.557]	0.693	0.250	46.9%
RF	Full	0.497 [0.465, 0.529]	0.477	0.438	0.495	−0.010 [−0.074, 0.053]	0.510 [0.473, 0.545]	0.694	0.250	44.3%

Table 14. Stage 2 test results (946 events, January–June 2024). Precision and recall are for the Long class. A model that always outputs 0.5 has log loss 0.693 and Brier score 0.250.

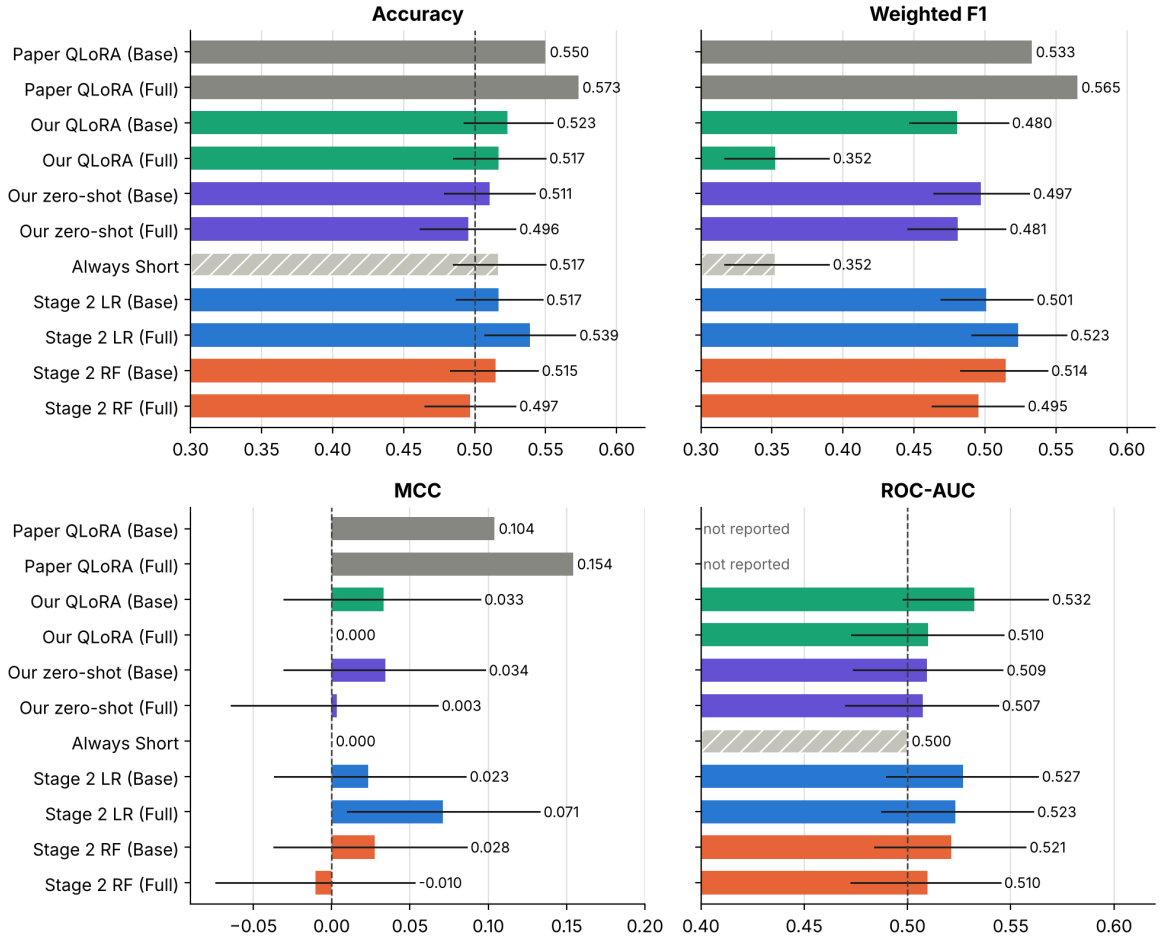


Figure 3. Test metrics of the paper, Stage 1, Always Short and Stage 2 models. Bars are point estimates, black lines 95% bootstrap intervals, dashed lines chance level.

7.3 Comparison with Stage 1

Model	Type	Data	Accuracy	Wtd F1	MCC	ROC-AUC	Compute
Paper QLoRA	LLM 8B	Base / Full	0.550 / 0.573	0.533 / 0.565	0.104 / 0.154	–	not reported
Our QLoRA (Stage 1)	LLM 8B	Base	0.523	0.480	0.033	0.532	about 6.4 h on T4 GPU
Our QLoRA (Stage 1)	LLM 8B	Full	0.517	0.352	0.000	0.510	about 6.3 h training on T4
Our zero-shot (Stage 1)	LLM 8B	Base	0.511	0.497	0.034	0.509	about 1 h on T4
Our zero-shot (Stage 1)	LLM 8B	Full	0.496	0.481	0.003	0.507	about 1 h on T4
Logistic Regression (Stage 2)	tabular	Base	0.517	0.501	0.023	0.527	all 32 runs plus evaluation: about 3–4 min on CPU
Logistic Regression (Stage 2)	tabular	Full	0.539	0.523	0.071	0.523	
Random Forest (Stage 2)	tabular	Base	0.515	0.514	0.028	0.521	
Random Forest (Stage 2)	tabular	Full	0.497	0.495	−0.010	0.510	
Always Short	reference	–	0.517	0.352	0.000	0.500	none

Table 15. All models on the same 946 test events.

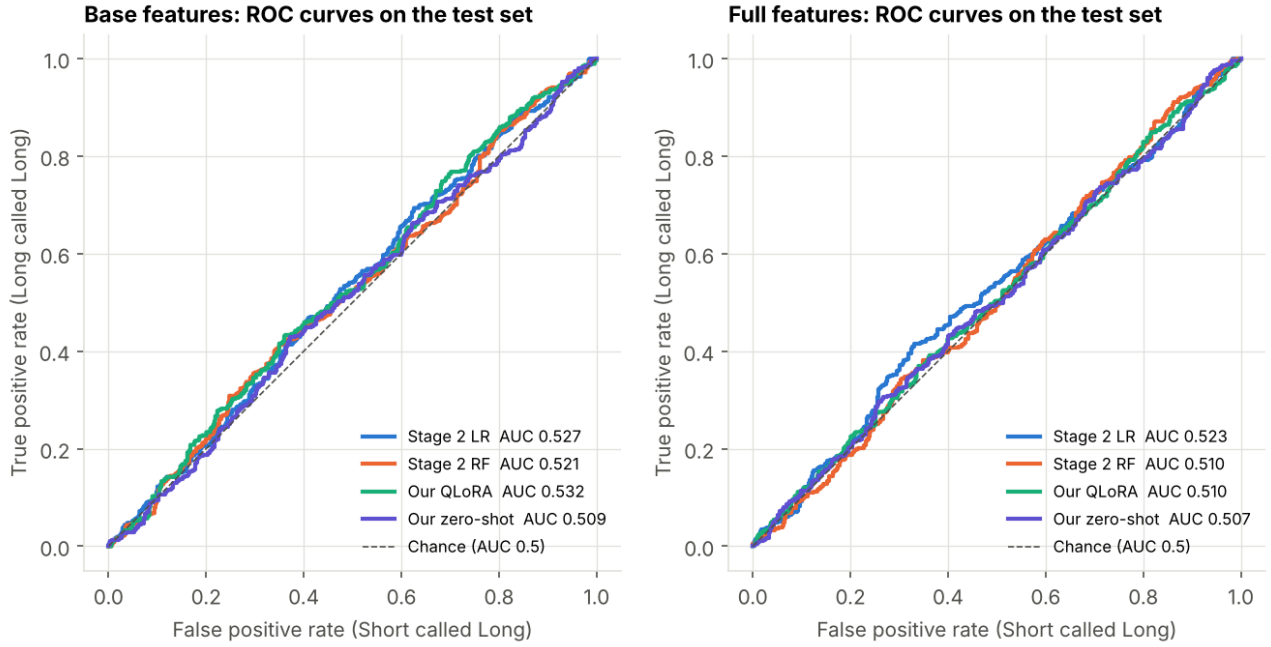


Figure 4. ROC curves on the test set. All curves stay close to the diagonal (chance).

7.4 Statistical tests

Model A	Model B	ΔAUC (A − B)	95% CI	p
LR Base	Our QLoRA Base	−0.005	[−0.057, 0.048]	0.84
LR Base	Our QLoRA Full	+0.017	[−0.037, 0.071]	0.52
LR Base	Always Short	+0.027	[−0.010, 0.063]	0.14
LR Full	Our QLoRA Base	−0.009	[−0.060, 0.044]	0.74
LR Full	Our QLoRA Full	+0.013	[−0.040, 0.066]	0.61
LR Full	Our zero-shot Full	+0.016	[−0.033, 0.066]	0.51
LR Full	Always Short	+0.023	[−0.012, 0.061]	0.19
RF Base	Our QLoRA Base	−0.011	[−0.064, 0.039]	0.65
RF Full	Our QLoRA Full	−0.000	[−0.051, 0.048]	0.98
LR Full	LR Base	−0.004	[−0.022, 0.017]	0.71
RF Full	RF Base	−0.011	[−0.043, 0.019]	0.45
LR Base	RF Base	+0.006	[−0.031, 0.044]	0.75
LR Full	RF Full	+0.013	[−0.021, 0.052]	0.46

Table 16. Paired bootstrap tests of ROC-AUC differences (2,000 resamples, seed 42): 13 key comparisons of the 24 run. All 24 are shown in Figure 5; none has $p < 0.05$ (smallest $p = 0.14$).

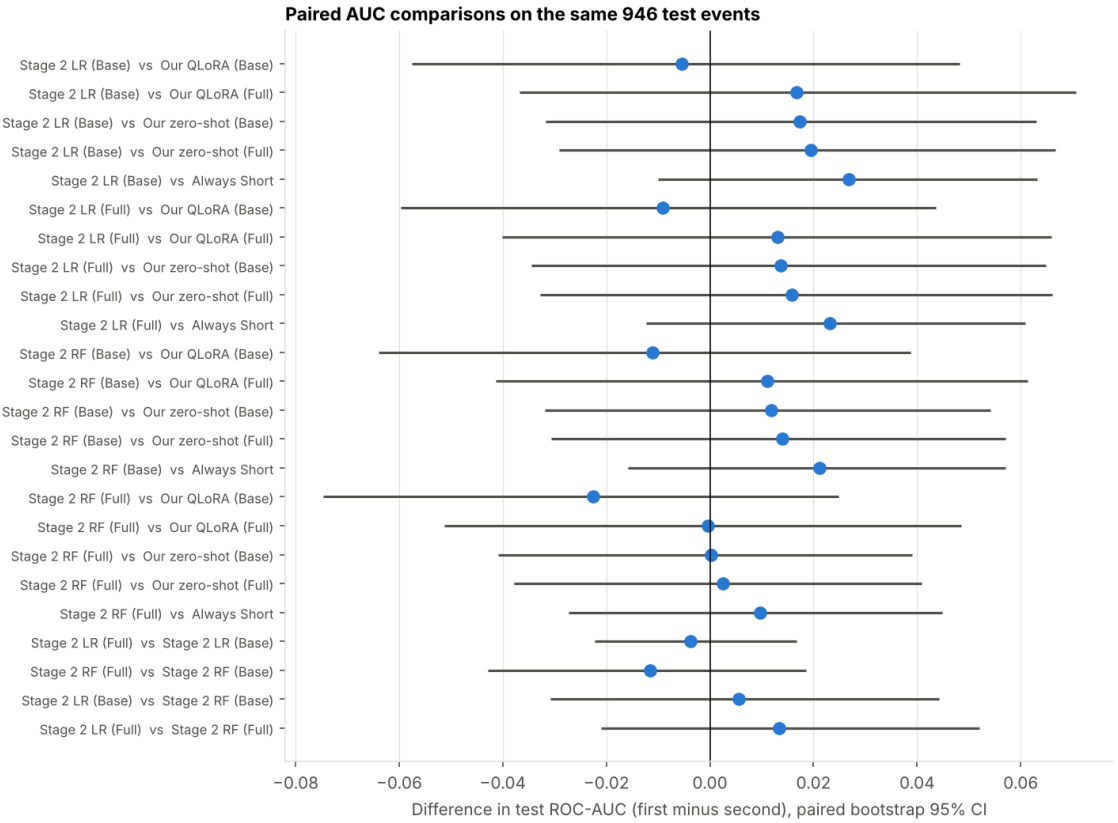


Figure 5. All 24 paired bootstrap differences in test ROC-AUC with 95% intervals. Every interval crosses zero.

7.5 Feature analysis

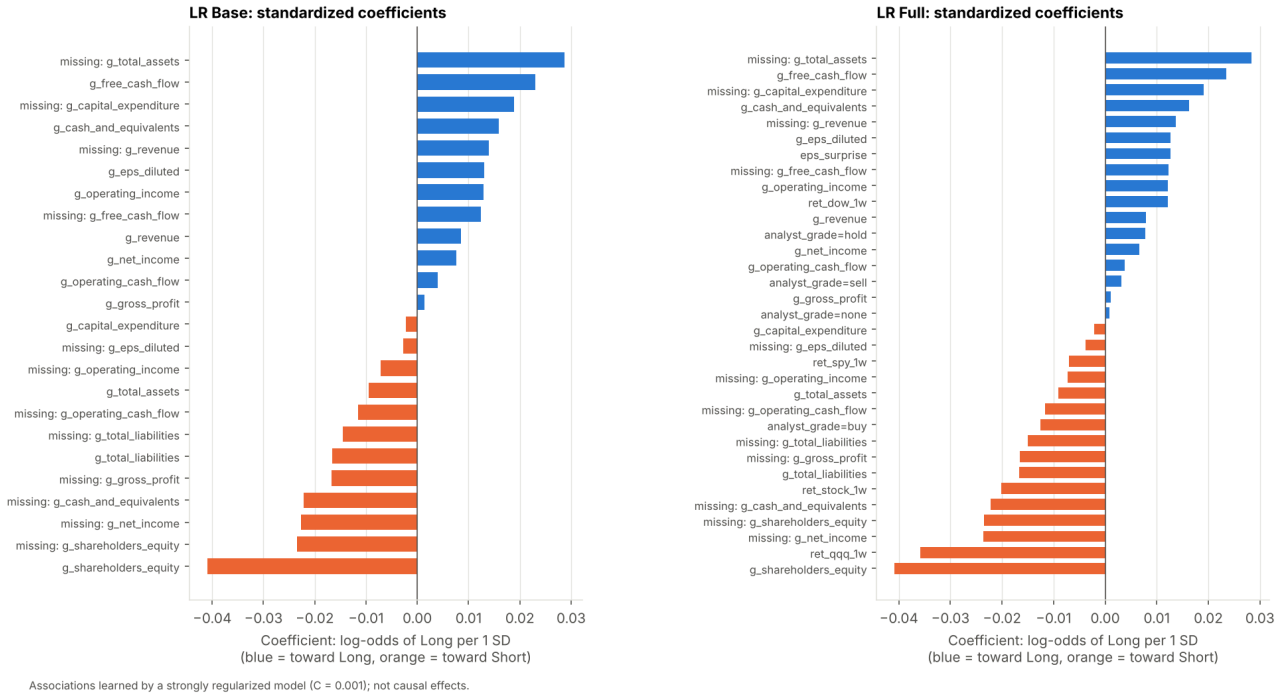


Figure 6. Standardized Logistic Regression coefficients (log-odds of Long per one standard deviation). Associations, not causes.

All coefficients are below 0.05 in absolute value. For comparison, a coefficient of 0.04 changes P(Long) by only about one percentage point when the feature moves by one standard deviation. The largest coefficients belong to Shareholders' Equity growth (toward Short) and several missing-value flags. In the Random Forest, permuting QQQ returns or revenue growth on validation slightly *increased* AUC, which suggests that splits on these features fit noise.

7.6 Observations

1. **No reliable signal.** All Stage 2 test AUCs (0.510–0.527) have 95% intervals that include 0.5, and validation AUCs were 0.486–0.520.
2. **No model is significantly better than another.** None of the 24 paired tests rejects equal AUC, including the tests against Always Short.
3. **LR Full accuracy 0.539 should not be over-read.** It is the best accuracy among our models and its MCC interval [0.010, 0.133] just excludes 0, but its AUC interval includes 0.5, its validation AUC was below chance (0.492), and with many intervals reported, one or two are expected to exclude zero by chance alone.
4. **No mode collapse.** Unlike the Full LLM, Stage 2 models predict both classes (32–47% Long), so their MCC and weighted F1 are meaningful.
5. **Same level at far lower cost.** Stage 2 matches the fine-tuned LLM's ranking ability with minutes of CPU time instead of hours of GPU time.
6. **External factors did not help** in either stage (Full versus Base: ΔAUC −0.004 for LR and −0.011 for RF, both not significant).

8 Conclusions

8.1 Main findings

1. **The paper's headline result was not reproduced** under a strict chronological split and free-GPU limits. Our fine-tuned Llama-3-8B reached 0.523 (Base) and 0.517 (Full) test accuracy with AUC 0.51–0.53. The paper's Base accuracy lies inside our confidence interval; its Full accuracy does not.
2. **Part of the paper was reproduced:** our zero-shot Llama-3 (0.496 on Full) matches the paper's GPT-4 baseline (0.494).
3. **The numbers alone carry no detectable signal** for next-day open-to-close direction: Logistic Regression and Random Forest on the same features reach AUC 0.510–0.527, statistically indistinguishable from the LLM and from Always Short.
4. Therefore the Stage 1 result is better explained by a **weak or absent signal in the inputs for this target** than by the LLM failing to read the numbers.

8.2 Improvements made

No modification improved predictive performance significantly, and we do not claim one. The improvements are methodological:

- A free, documented rebuild of the paper's dataset (96.7% of its size) with a leakage-free time split.
- Detection and correction of the fp16 tie problem, which inflated weighted F1 by about 7 points.
- Extractive summarization that fits whole calls into 3,072 tokens.
- Probability outputs, ROC-AUC, confidence intervals and paired significance tests, none of which the paper reports.
- Interpretable numerical models that match the LLM at a tiny fraction of the compute.

8.3 Limitations

- Context of 3,072 tokens and 3,000 training events in Stage 1, forced by the free GPU.
- Free data sources: features, transcripts and the company set differ from the paper's.
- A single chronological split with a test period of six months (946 events); results for other periods may differ.
- The label ignores the overnight price jump right after the announcement, where much of the earnings reaction may happen; the remaining open-to-close move may be mostly noise (our hypothesis, not tested).

8.4 Future work

- Test other targets: the overnight gap, close-to-close or multi-day returns, or returns in excess of the market.
- Add richer numbers, such as estimate revisions and the size of analyst disagreement.
- Combine numerical models with compact text embeddings (e.g. FinBERT) to test whether the transcript adds information.

8.5 Learning outcomes

- How to build a financial dataset from public APIs without look-ahead bias.
- How QLoRA fine-tunes an 8-billion-parameter model on a 16 GB GPU, and how numeric precision (fp16 versus fp32) can change reported metrics.
- Why time-based splits, validation-only model selection and a single test evaluation are essential in finance.
- How to report results honestly with confidence intervals, paired tests and simple baselines such as Always Short.

9 References

1. H. Ni, S. Meng, X. Chen, Z. Zhao, A. Chen, P. Li, S. Zhang, Q. Yin, Y. Wang, and Y. Chan, "Harnessing Earnings Reports for Stock Predictions: A QLoRA-Enhanced LLM Approach," arXiv:2408.06634v2 [q-fin.CP], 2024.
2. T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems* 36 (NeurIPS), 2023.
3. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations* (ICLR), 2022.
4. Meta AI, "The Llama 3 Herd of Models," arXiv:2407.21783, 2024.
5. D. Han, M. Han, and the Unsloth team, "Unsloth: faster, memory-efficient LLM fine-tuning," https://github.com/unslothai/unsloth.
6. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, 12, pp. 2825–2830, 2011.
7. L. Breiman, "Random Forests," *Machine Learning*, 45(1), pp. 5–32, 2001.
8. B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall/CRC, 1993.
9. B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," *Biochimica et Biophysica Acta*, 405(2), pp. 442–451, 1975.
10. U.S. Securities and Exchange Commission, "EDGAR Application Programming Interfaces (companyfacts)," https://www.sec.gov/edgar/sec-api-documentation.
11. kurry, "sp500_earnings_transcripts," Hugging Face dataset, https://huggingface.co/datasets/kurry/sp500_earnings_transcripts.
12. R. Aroussi, "yfinance: Yahoo! Finance market data downloader," https://github.com/ranaroussi/yfinance.
13. datasets/s-and-p-500-companies, "S&P 500 companies list," GitHub, commit 9b6713a3 (10 Aug 2024), https://github.com/datasets/s-and-p-500-companies.

Appendix A How to Reproduce

All files are in the project folder; `README.md` files in each folder describe every file.

Folder	Contents
`0_paper/`	The original paper (PDF)
`stage1/1_data/`	`llm_data.jsonl.gz` (8,271 events), `llm_data_sum.jsonl.gz` (summarized transcripts)
`stage1/2_code/`	`dataset_builder/step01–05`, `llm_stage1.py`, `summarize_transcripts.py`, tests
`stage1/3_notebooks/`	`1_Train_and_Evaluate.ipynb` (Kaggle training and scoring), `2_Predict_New_Transcript.ipynb`
`stage1/4_results/`	Per-event test predictions and `summary_all_runs.json`
`stage2/src/`	Extraction, verification, preprocessing, training, evaluation, plots, checks, demo
`stage2/results/`	Predictions, metrics, fitted models (`.joblib`), figures

Stage 1 (Kaggle, T4 GPU). Upload `llm_data_sum.jsonl.gz` and `llm_stage1.py` as Kaggle datasets, open `1_Train_and_Evaluate.ipynb`, set `DATASET = "base"` or `"full"` and `MODE = "train_eval"` or `"zeroshot"`, and run all cells. One training run takes about 6–7 hours.

Stage 2 (any CPU, about 3–4 minutes). From the project root:

```
pip install -r stage2/requirements.txt
python stage2/src/run_all.py
python stage2/src/predict_demo.py --dataset full --event NVDA_2024-02-21
```

`run_all.py` rebuilds the features, trains and selects all models, evaluates them, draws the figures, and runs `final_checks.py`, which asserts identical test IDs and labels across all stages and re-computes the Stage 1 metrics from the saved prediction files. Re-running the full pipeline from scratch reproduced every reported number.